9 Sharding and persistence

This chapter covers

- Distributing actors in a cluster
- Storing each change of the actor's state in a database

This chapter discusses two of the seven tools used to build microservices: Akka Sharding and Akka Persistence. The others are Akka Streams, Persistence Query, Projections, HTTP, and gRPC, which you learn about in chapters 10 and 11.

These Akka tools can be used separately (and are traditionally used that way). However, they are increasingly used as a functional unit: looking at them all together helps us better understand them. We call this functional view *Akka microservices* (to learn more, see appendix B).

NOTE The source code for this chapter is available at www.manning.com/books/akka-in-action-second-edition or https://github.com/franciscolo pezsancho/akka-topics/tree/main/chapter09a. You can find the contents of any snippet or listing in the .scala file with the same name as the class, object, or trait.

9.1 Akka sharding and stateful systems

Akka cluster sharding distributes the locations of actors; they are called *entities* when sharding. With cluster sharding, you can ensure that each entity is unique and that there is at most one in the cluster at any given time.

DEFINITION *Shards* are logical containers in which entities are grouped.

Entities are automatically relocated when new nodes are added or current nodes leave the cluster. This is primarily useful for two reasons. Perhaps most obvious is the sharing of resources so the memory required by the actors is distributed among the nodes.

NOTE The node of an entity is selected by the modulus of its ID hashed over the number of shards.

Relocating entities also helps in case of failures. The entities are protected when a node fails by moving them to another healthy member of the cluster. Isolation by node also protects entities that live in healthy nodes from failures of other nodes. However, when an entity is moved, it loses its state because the entity is restarted. To preserve its state, you need a storage mechanism: Akka Persistence, which we discuss later in this chapter.